

Foundations of Data Science for Biologists

Programming with *AWK*

BIO 724D

2026-FEB-03

Instructors: Greg Wray and Kayla Wilhoit

Learning goals for today

learn how to compose useful 1-line awk commands

learn how to write programs using awk

Introducing *AWK*

AWK is a programming language that is designed to work efficiently on tabular data based on pattern matching.

AWK is named after its inventors, three computer scientists at Bell Labs in the 70s:

- **Alfred V. Aho**: also invented efficient algorithms for regular expressions that were critical for their adoption
- **Peter J. Weinberger**: contributed to AWK and modern Fortran; became head of computer science research at Bell Labs in the 80s
- **Brian W. Kernighan**: major contributor to the development of Unix; helped to popularize the C programming language (Kernighan and Ritchie, The C Programming Language, 1978 is known simply as “K&R C”)

Why use AWK?

AWK combines the text-processing powers of many of the core Unix utilities such as cut, grep, and sed into a single, flexible programming language.

AWK is concise and has a relatively simple syntax.

AWK integrates seamlessly into Unix/Linux pipelines and scripts.

AWK is fast and can handle GBs of data very efficiently!

AWK is nearly universal: anyone with access to Unix/Linux can run your program.

Note the parallels with SED

sed:

```
sed options address 'command' input.txt
```

awk:

```
awk options 'pattern {action}' input.txt
```

pattern in awk is similar to **address** in sed, but is more flexible

action in awk is similar to **command** in sed, but can be any arbitrary computation

May include multiple commands with variables, functions, control flow

Supports numeric, string, and associative array (dictionary)

One-line commands with awk

Basic syntax of an awk command

Basic syntax

```
awk 'pattern { action }' input_file
```

separate multiple actions with ";"

Example

```
awk '$5 > 42 { print $0 }' input_file
```

Omitting the **pattern** sets it to true, thus applying the action to every line

```
awk '{ print $5,$2 }' input_file
```

prints columns 5 and 2

Omitting the **action** sets it to the default action, which is to print the entire line

```
awk ' NR == 3 ' input_file
```

prints line 3

AKW one-liners: printing

```
{ print NR, $0 }
```

```
{ print $1, $7, $2 }
```

```
{ print $NF }
```

```
{ $2=''; print }
```

```
{ $1=NR; print }
```

```
{ tf=$1; $1=$2; $2=tf; print }
```

```
NF > 0
```

```
NF > 4
```

```
$9 > 200
```

number each line, then print

print only specified columns, in order given

print only last column

delete 2nd column, then print

replace 1st column with line number, then print

swap first two columns, then print entire line

print non-empty lines

print lines with more than 4 columns

print lines with value > 200 in column 9

Substitute the code into: `awk -F", " '{<code>' seg_strain_table.csv`

AKW one-liners: filtering using regex

```
' /^"PMY/ '
```

```
' $0 !~ /^"PMY/ '
```

```
' $5 ~ /^N/ '
```

```
' $5 !~ /^N/ '
```

```
' /^"PMY124/ || $6 > 3 '
```

```
' /^"PMY124/ && $6 > 3 '
```

print lines containing a match to a regex

print lines not matching a regex

print lines matching a regex in column 5 only

print lines not matching a regex in column 5

print lines matching regex or condition (OR)

print lines matching both regex (AND)

Substitute the code into: `awk -F", " ' <code> ' seg_strain_table.csv`

AKW one-liners: adding columns

```
'{ $(NF+1) = 42; print }'
```

```
'{ print $1, $12+$13+14/3 }'
```

```
'{ if($7>$8) {$15="T"}; print }'
```

```
'{ print $0, length($6) }'
```

add a new last column with a fixed value

add a new column with a computed value

add a new column based on condition

add a new column with return from a function

Substitute the code into: `awk -F", " '{<code>' seg_strain_table.csv`

AKW one-liners: text substitution, deletion, and extraction

```
'{ sub(/N./,"missing"); print }'
```

substitute first match only on each line

```
'{ gsub(/N./,"missing"); print }'
```

substitute all matches

```
'{ gsub(/N./,"-99", $5); print }'
```

substitute matches in column 5 only

```
'{ gsub(/"/,""); print }'
```

delete all double-quote characters

```
'{ print substr(5,4) }'
```

extract characters 5-9 from the entire line

```
'{ print substr($1,5,4) }'
```

extract characters 5-9 from column 1

Substitute the code into: `awk -F"," '{<code>}' seg_strain_table.csv`

AKW one-liners: text manipulation

```
'{ print tolower($0) }'  
'{ print toupper($2) }'  
'{ $2 = toupper($2); print }'  
'{ $1 = "ID_"$1; print }'  
'{ print $1, substr($12,1,4) }'
```

convert all string values to lower case

print only column 2, converted to upper case

same as above, but print entire line

add prefix to values in column 1

trim values in column 12 to 2 decimal places

Substitute the code into: `awk -F", " '{<code>' seg_strain_table.csv`

Applying awk one-liners to a real-world problem

Application: processing protein sequence alignments

The alignment file is in FASTA format:

```
>Corynebacterium_sputi
MVGFDSTDNDGTTEMD-AVPQDFNWVWQEVIQLVAASDDP-----MRPDLTLRASDRSLLASVRPVGQVNGYVL...
>Dietzia_natronolimnaea
-----MSPDLEHVWSAVLDKLTDP SLA-----SDDNPALSRQQQAWLRLVQPIAVVNGFAM...
>Gordonia_iterans
-----MADEHDSFSAIWSRVVAELSDAAP-----PDSPPLSRQQKAWLSLVHPLTITSGFAL...
```

We want:

- Plain-text file

- Each accession on one line, with alignment maintained and the > character removed

- Amino acids on either side of a focal region removed while retaining the taxon labels

Convert alignment FASTA to text file and pad taxon names to maintain alignment

Convert FASTA to .tsv format

```
seqkit fx2tab job_number.fa > aln_full.tsv
```

Pad taxon names to 35 spaces with "_" and convert to .csv format

```
awk -F'\t' -v OFS=',' '{ $1 = sprintf("%-35s", $1); gsub(/ /, "-", $1); \
print }' aln_full.tsv > aln_full.csv
```

Remove the comma: two equivalent commands

```
sed 's/,//g' aln_full.csv > aln_full.txt
```

```
awk '{ sub(/,/,""); print }' aln_full.csv > aln_full.txt
```

Remove amino acids on either side of focal region while retaining taxon labels

Trim amino acids after domain 1

```
awk -v len=170 '{print substr($0, 1, len)}' aln_full.txt > aln_d1.txt
```

Trim amino acids from start of protein sequence to beginning of core domain 1

```
awk '{ print substr($0, 1, 35) substr($0, 65) }' aln_d1.txt > aln_d1_s.txt
```

Programming with awk

Simple multiline programs

Program to print a file, skipping comment lines and empty lines:

```
/^#/ { next }           # skip comment lines  
NF > 0                 # print non-blank lines
```

Program to calculate number of lines and average line length:

```
{ linesum += length($0) }      # tally line lengths  
END { print NR, linesum/NR }   # when done, print values
```

Note: single quotes are not needed; each command goes on a separate line

Saving and running an awk program

Write your program in interactive mode, testing each step as you go

Open a new file in a text editor and type in your program (see next slide for format)

Save your file; using the extension `.awk` is recommended for clarity

To run your program:

```
awk -f my_program.awk input_file
```

Structure of an awk program

```
# optional BEGIN block is evaluated before lines are processed
BEGIN {
    initial actions           # common uses: setting separators, initializing variables
}

# main block is an implicit loop that contains one or more commands

# place multiple pattern-action commands on separate lines
pattern { action }
pattern { action }

# commands can be nested, e.g., for control flow
pattern {
    pattern { action }
}

# optional END block is evaluated after lines are processed
END {
    final actions            # common uses: reporting values and exit status
}
```

Variables

Variables types are automatically determined at assignment:

```
'{ msg="hello" $1; print msg }'
```

prints a string

```
'{ msg=42+$7; print msg/3 }'
```

prints a number

awk also offers many useful built-in variables:

```
'{ print "this line has " NF "fields }'
```

reports the number for each line

```
'{ print "processing " FILENAME }'
```

reports the name of the file

See the reference slides for many more built-in variables

Substitute the code below into: `awk -F", " '<code>' seg_strain_table.csv`

Associative arrays

```
# tally the frequency of the first word on each line

{ count[$1]++ }          # tallies values from column 1

END {                   # display the associative array
    for (word in count) { print word, count[word] }
}
```

Field separators; field and record counts

Field separators can be specified

FS = input field separator (default any whitespace)

OFS = output field separator (default = single space)

```
BEGIN {  
    FS = "\t"      # input is TAB-delimited (.tsv)  
    OFS = ","      # output is comma-delimited (.csv)  
}
```

Record and field counts are dynamically updated

NR and FNR = number of records; FNR is current when multiple files are submitted

NF = number of fields (columns) in the current line

```
awk -F"\t" '{ print "There are " NF "fields in line " NR }' input.txt
```

Applying an awk program to a real-world problem

Example of short program

```
# program that counts the frequency of each word in a file

# first, set the field separator to be any non-printing character
BEGIN { FS="^[^a-zA-Z]+" }

# loop through words on the current line
{ for (i=1; i<=NF; i++)
    { words[tolower($i)]++ }
}

# output the contents of the associative array
END { for (word in words) { print word, words[word] } }
```

Example of a longer program

```
# tally the number of genes and compute average gene length from a .gff file

BEGIN {
    FS = "\t"           # Tab is the field delimiter in input files
    gene_ct = 0         # initialize a gene counter variable
    size_sum = 0        # initialize a variable to hold the sum of gene lengths
}

/^##FASTA/ { exit }    # end of tabular GFF features; exit and don't process more lines

/^#/ { next }          # skip comment lines

NF != 9 { next }       # validate that 9 fields are present; skip if not

$3 == "gene" {         # if the feature is a gene:
    gene_ct += 1        # increment gene count by 1
    gene_size = $5 - $4 + 1 # calculate gene length
    size_sum += gene_size # increment total gene length
}

END {                  # report results
    print "Number of genes: " gene_ct
    print "Average gene size: " size_sum / gene_ct
}
```

Reference

Using awk

Interactive mode:

```
awk options 'pattern { action }' input_file
```

Enclose the program in single quotes

Can include multiple pattern-action steps and span multiple lines

Multiple input files can be provided; separate file names with spaces

Using a stored script:

```
awk -f my_program.awk input_file  
my_program.awk input_file
```

for a normal text file

for an executable file

Common options for awk

- | | | |
|----|--------------------------------|--|
| -f | read program from file | <code>awk -f myprog.awk input.txt</code> |
| -F | specify field separator | <code>awk -F"," myprog.awk input.txt</code> |
| -v | assign variable before program | <code>awk -v myvar=1 myprog.awk input.txt</code> |

Useful built-in variables (see documentation for a full list)

<code>\$0</code>	the entire current line
<code>\$n</code>	the nth field (column) of the current line; e.g., <code>\$1</code> , <code>\$2</code> , <code>\$3</code> , etc.
<code>\$NF</code>	the last field of the current line
<code>NR</code>	number of record: the current line number being processed (0-based)
<code>FNR</code>	as above, but resets to <code>0</code> for each file; for working with multiple files
<code>NF</code>	number of fields: the number of fields (columns) in the current line
<code>FS</code>	input field separator (default = space)
<code>RS</code>	input record separator (default = newline)
<code>OFS</code>	output field separator (default = space)
<code>ORS</code>	output record separator (default = newline)
<code>FILENAME</code>	name of the file currently being processed; undefined in the <code>BEGIN</code> block
<code>ARGC</code>	count of variables (tokens?) at the command line; includes awk itself
<code>ARGV</code>	array containing command line variables; <code>awk=0</code> , index by integer in order
<code>RLENGTH</code>	length of string matched by awk's <code>match</code> function
<code>RSTART</code>	first position in the string matched by awk's <code>match</code> function

Useful built-in functions for awk programs (see documentation for a full list)

`print($n)`

print field (column); default value `$0`

`printf($n, expr-list)`

print formatted; default value `$0`

`gsub(regex, str)`

carries out global substitution of a regex with a string

`match(str, regex)`

returns the index of a regex in a string

`int(num)`

truncates floating point to integer (does not round!)

`tolower(str)`

converts string to all lower case

`toupper(str)`

converts string to all upper case

`strtonum(str)`

converts numerals-only string to numeric

`length(str)`

returns the length of a string

`function name {action}` defines a function

